

Chapter 05 · SQL Databases

Sub-Chapter 10 — PostgreSQL vs MySQL

10 / 10

Scope	Feature comparison, architecture, SQL differences, Spring config
Engines	PostgreSQL 16 vs MySQL 8.0
Level	Intermediate
Outcome	Make an informed choice and configure Spring for either DB

1 PostgreSQL vs MySQL — The Context

Both are excellent; the difference is in the details

PostgreSQL and MySQL are the two most-used open-source relational databases. Both support ACID transactions, indexes, stored procedures, and replication. The differences emerge in **advanced SQL features**, **data types**, **architecture**, and **ecosystem**. This chapter is a practical comparison for a Java/Spring backend engineer choosing between them — or maintaining both.

PostgreSQL vs MySQL — Feature Comparison

Feature	PostgreSQL	MySQL
ACID / Transactions	Full ACID, all isolation levels	Full ACID (InnoDB only)
JSON support	JSONB with GIN index, operators	JSON column, limited ops
Full-text search	Built-in tsvector/tsquery	Basic FULLTEXT index
Window functions	Full SQL:2003 support	Since 8.0 (limited)
CTEs (recursive)	Yes, since 8.4	Since 8.0
Materialized views	Yes, REFRESH CONCURRENTLY	No (workaround needed)
Partitioning	Declarative (RANGE/LIST/HASH)	RANGE/LIST/COLUMNS/HASH
Foreign keys	Full support	InnoDB only, limited
Stored procedures	PL/pgSQL, PL/Python, PL/Perl...	Since 8.0 (basic)
Replication	Streaming, logical, slots	Binlog replication, GTID
Extensions	PostGIS, pgvector, pg_cron, 300+	Limited plugin ecosystem
Licensing	PostgreSQL License (permissive)	GPL v2 / commercial

2 Architecture Differences

Process model, storage engines, MVCC implementation

Architecture Differences at a Glance

PostgreSQL

Process per connection (fork)
MVCC with heap + dead tuples + VACUUM

Storage: single heap file per table
WAL for crash recovery & replication
Extensions via shared libs (PostGIS etc.)
Object-relational: custom types, operators

MySQL

Thread per connection (lighter weight)
MVCC with undo log segments (InnoDB)

Storage: clustered index (B+Tree by PK)
Binary log (binlog) for replication
Plugin-based storage engines (InnoDB default)
Relational only (no custom types)

MySQL's clustered index means PK lookups are fast; secondary index lookups require 2 B-Tree traversals
PostgreSQL's heap means all indexes are equal — any index can find a row in one lookup

2.1 MVCC Implementation

Both databases use **Multi-Version Concurrency Control (MVCC)** to allow readers and writers not to block each other, but they implement it differently:

PostgreSQL — Old row versions (dead tuples) are stored in the same heap file. VACUUM periodically cleans them. This means VACUUM is essential; a blocked VACUUM causes table bloat.

MySQL/InnoDB — Old versions are stored in a separate undo log. No VACUUM needed. But the undo log can grow large under long-running transactions and history list length (HLL) must be monitored.

2.2 Clustered vs Heap Storage

```
-- MySQL InnoDB: table IS the B+Tree indexed by primary key -- SELECT by PK: 1 B-Tree lookup → row data (fast) -- SELECT by secondary index: 2 lookups (secondary index → PK → row) SELECT * FROM orders WHERE id = 42; -- 1 lookup SELECT * FROM orders WHERE customer_id = 101; -- 2 lookups (sec idx + PK) -- PostgreSQL: all indexes point to the physical row location (ctid) -- SELECT by any index: 1 index lookup → heap page (equal cost) -- But: sequential scans on heap can be slower after many updates (bloat) SELECT * FROM orders WHERE id = 42; -- 1 index + heap SELECT * FROM orders WHERE customer_id = 101; -- 1 index + heap
```

3

SQL Feature Differences

Where PostgreSQL goes further than MySQL

3.1 JSONB — PostgreSQL's Killer Feature

```
-- PostgreSQL JSONB: stored binary, indexed with GIN CREATE TABLE events ( id BIGSERIAL PRIMARY KEY, payload JSONB NOT NULL ); CREATE INDEX idx_events_payload ON events USING GIN (payload); -- Query nested JSON fields with operators SELECT * FROM events WHERE payload @> '{"type": "ORDER_PLACED"}'; -- @> = contains SELECT payload -> 'user' -> 'email' AS email -- -> object, ->> text FROM events WHERE payload ? 'user'; -- ? = key exists -- MySQL JSON: no binary storage in older versions, fewer operators SELECT JSON_EXTRACT(payload, '$.user.email') FROM events; -- No GIN equivalent – only functional index on a single path
```

3.2 Materialized Views

```
-- PostgreSQL: real materialized views CREATE MATERIALIZED VIEW monthly_revenue AS SELECT DATE_TRUNC('month', created_at) AS month, SUM(total) AS revenue, COUNT(*) AS order_count FROM orders GROUP BY 1; -- Refresh without blocking reads (PostgreSQL 9.4+) REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_revenue; -- MySQL: no materialized views – common workaround: manually maintained table CREATE TABLE monthly_revenue_cache AS SELECT ...; -- static snapshot -- Must be refreshed manually or via event scheduler
```

3.3 LATERAL JOIN (PostgreSQL only until MySQL 8.0.14+)

```
-- PostgreSQL LATERAL: right side can reference columns from left side SELECT c.name, recent.* FROM customers c CROSS JOIN LATERAL ( SELECT total, created_at FROM orders WHERE customer_id = c.id -- references c.id ORDER BY created_at DESC LIMIT 3 ) recent; -- MySQL 8.0.14+ supports LATERAL with INNER JOIN / CROSS JOIN JOIN LATERAL
```

3.4 Table Inheritance (PostgreSQL only)

```
-- Useful for modelling entity hierarchies CREATE TABLE vehicles (id BIGSERIAL PRIMARY KEY, make TEXT, model TEXT, year INT); CREATE TABLE cars (num_doors INT) INHERITS (vehicles); CREATE TABLE trucks (payload_kg INT) INHERITS (vehicles); -- Query parent sees all children: SELECT * FROM vehicles; -- returns cars AND trucks -- MySQL alternative: single-table inheritance (discriminator column)
```

4

Spring / JPA Differences

Driver config, dialect, and driver-specific behaviours

4.1 Dependencies & Dialect

```
org.postgresql postgresql runtime com.mysql mysql-connector-j runtime
```

```
# application.yml – PostgreSQL spring: datasource: url: jdbc:postgresql://localhost:5432/mydb
username: app password: secret driver-class-name: org.postgresql.Driver jpa: database-platform:
org.hibernate.dialect.PostgreSQLDialect hibernate: ddl-auto: validate properties: hibernate:
default_schema: public --- # application.yml – MySQL spring: datasource: url:
jdbc:mysql://localhost:3306/mydb?useSSL=false&serverTimezone=UTC username: app password: secret
driver-class-name: com.mysql.cj.jdbc.Driver jpa: database-platform:
org.hibernate.dialect.MySQLDialect hibernate: ddl-auto: validate
```

4.2 AUTO_INCREMENT vs SEQUENCE

```
// PostgreSQL: @GeneratedValue(strategy = SEQUENCE) is most efficient // Uses a DB sequence –
Hibernate can batch-fetch IDs @Entity public class Order { @Id @GeneratedValue(strategy =
GenerationType.SEQUENCE, generator = "order_seq") @SequenceGenerator(name = "order_seq",
sequenceName = "order_id_seq", allocationSize = 50) // pre-fetch 50 IDs private Long id; } //
MySQL: @GeneratedValue(strategy = IDENTITY) – uses AUTO_INCREMENT // IDENTITY disables Hibernate's
JDBC batch inserts (Hibernate limitation) @Entity public class Order { @Id
@GeneratedValue(strategy = GenerationType.IDENTITY) private Long id; } // To regain batch inserts
with MySQL use: // spring.jpa.properties.hibernate.jdbc.batch_size=50 //
spring.jpa.properties.hibernate.order_inserts=true
```

4.3 Case-Sensitivity Quirk

MySQL on Windows/Mac is case-insensitive for table names by default (`lower_case_table_names=1`). PostgreSQL folds unquoted identifiers to lowercase. This means a Flyway migration that creates `UserProfile` in MySQL works but in PostgreSQL it creates `userprofile`. Always use lowercase snake_case for table and column names — works on both.

5 Migration Between the Two

Moving from MySQL to PostgreSQL (and back)

Many teams start on MySQL and migrate to PostgreSQL as their needs grow. The main migration challenges are SQL dialect differences and data type mappings.

5.1 Common SQL Syntax Differences

Feature	MySQL	PostgreSQL
Auto-increment	AUTO_INCREMENT	SERIAL or GENERATED ALWAYS AS IDENTITY
String concat	CONCAT(a, b) or a + b (wrong!)	a b or CONCAT(a, b)
LIMIT/OFFSET	LIMIT 10 OFFSET 20	LIMIT 10 OFFSET 20 (same)
Backtick quoting	`column_name`	"column_name" (double quotes)
Boolean	TINYINT(1) or BIT(1)	BOOLEAN (native)
UUID	CHAR(36) or BINARY(16)	UUID (native type)
NOW()	NOW() or SYSDATE()	NOW() or CURRENT_TIMESTAMP
Regex	REGEXP or RLIKE	~ (POSIX) or SIMILAR TO

UPSERT	INSERT ... ON DUPLICATE KEY UPDATE	INSERT ... ON CONFLICT DO UPDATE
JSON column	JSON (text storage pre-8.0)	JSONB (binary, indexed)

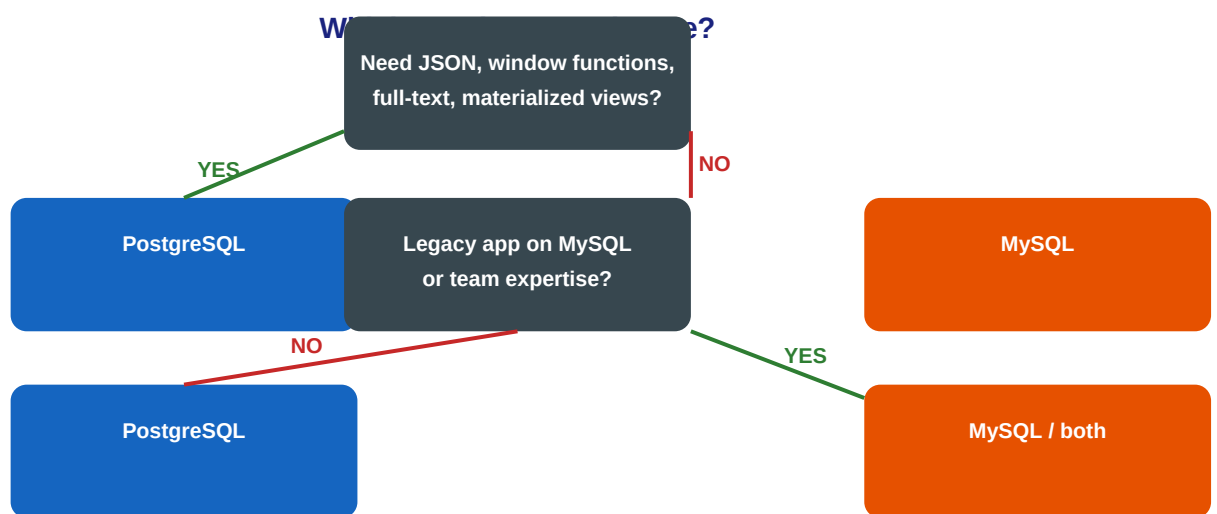
5.2 Flyway — Database-Agnostic Migrations

```
# application.yml – Flyway picks the right location automatically
spring:
  flyway:
    locations:
      classpath:db/migration/{vendor}
# For PostgreSQL: uses db/migration/postgresql/
# For MySQL: uses db/migration/mysql/
# Shared SQL: db/migration/ (vendor-neutral SQL)
# Directory structure:
# src/main/resources/db/migration/
# V1__create_tables.sql (works on both)
# postgresql/
# V2__add_jsonb_column.sql (PostgreSQL-specific)
# mysql/
# V2__add_json_column.sql (MySQL-specific)
```

6

Decision Guide

When to pick PostgreSQL, when MySQL is fine



When in doubt, default to PostgreSQL — richer SQL, better extensibility, no licensing surprises

Choose PostgreSQL when:

- You need JSONB, full-text search, materialized views, or complex window functions.
- Your domain involves geospatial data (PostGIS extension).
- You want logical replication or CDC (change data capture) with replication slots.
- You need custom types, operators, or procedural languages (PL/Python, PL/R).
- You want pgvector for storing ML embeddings alongside relational data.
- You are starting a new project with no legacy constraints.

MySQL is a solid choice when:

- Your team has deep MySQL expertise and existing tooling.
- You are running on AWS RDS Aurora MySQL (excellent managed service).
- Your application is a simple CRUD app without advanced SQL needs.
- You need specific MySQL features like Group Replication or NDB Cluster.

7

Quick-Reference Cheatsheet

Side-by-side syntax for everyday Spring/JPA tasks

Task	PostgreSQL	MySQL
Driver artifact	org.postgresql:postgresql	com.mysql:mysql-connector-j
JPA Dialect	PostgreSQLDialect	MySQLDialect
ID generation	SEQUENCE (allocationSize=50)	IDENTITY (disables batching)
UPSERT	ON CONFLICT (col) DO UPDATE SET ...	ON DUPLICATE KEY UPDATE ...
JSON query	payload @> '{}' or ->> operator	JSON_EXTRACT(col, '\$.path')
Regex match	col ~ 'pattern'	col REGEXP 'pattern'
Current time	NOW() / CURRENT_TIMESTAMP	NOW() / SYSDATE()
Boolean column	BOOLEAN	TINYINT(1)
UUID type	UUID (native)	CHAR(36) or BINARY(16)
Table info (psql)	\d tablename	DESCRIBE tablename
Case sensitivity	Folds unquoted to lowercase	Windows/Mac: case-insensitive